S.No: 1	Exp. Name: ***Program to implement AVL tree and its operations***	**Date: 2026-10-01**

Aim:

Write a program to create an AVL tree using a provided set of elements stored in a file. Implement operations for inserting and deleting elements from the AVL tree. After constructing the tree, output the contents of the tree in in-order traversal to a new file.

Note: Refer to visible test cases for better understanding.

Source Code:

avl_tree.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
typedef struct Node {
    int key;
    struct Node* left;
    struct Node* right;
    int height;
} Node;
// Utility functions
int height(Node* N) {
    if (N == NULL) return 0;
    return N->height;
}
int max(int a, int b) {
    return (a > b) ? a : b;
}

Node* newNode(int key) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->key = key;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}
//example insert 50(50 is key ,its height is initialized with 1)
// Rotations
//LL imbalance
Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;
    return x;
}
Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = max(height(x->left), height(x->right)) + 1;
```

```
    y->height = max(height(y->left), height(y->right)) + 1;
    return y;
}
int getBalance(Node* N) {
    if (N == NULL) return 0;
    return height(N->left) - height(N->right);
}
// Insert
Node* insert(Node* node, int key) {
    if (node == NULL)
        return newNode(key);
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node; // No duplicates
    node->height = 1 + max(height(node->left), height(node->right));
    int balance = getBalance(node);
    // Balancing
//LL IMBALANCE
    if (balance > 1 && key < node->left->key) return rightRotate(node);
//RR imbalance
    if (balance < -1 && key > node->right->key) return
leftRotate(node);
//LR IMBALANCE
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
//RL IMBALANCE
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}
// Minimum value node
Node* minValueNode(Node* node) {
    Node* current = node;
    while (current->left)
        current = current->left;
    return current;
}
// Delete
Node* deleteNode(Node* root, int key, int* found) {
```

```
    if (!root) return root;
    if (key < root->key)
        root->left = deleteNode(root->left, key, found);
    else if (key > root->key)
        root->right = deleteNode(root->right, key, found);
    else {
        *found = 1;
        if (!root->left || !root->right) {
            Node* temp = root->left ? root->left : root->right;
            if (!temp) {
                temp = root;
                root = NULL;
            } else
                *root = *temp;
            free(temp);
        } else {
            Node* temp = minValueNode(root->right);
            root->key = temp->key;
            root->right = deleteNode(root->right, temp->key, found);
        }
    }
    if (!root) return root;
    root->height = 1 + max(height(root->left), height(root->right));
    int balance = getBalance(root);
    if (balance > 1 && getBalance(root->left) >= 0) return
rightRotate(root);
    if (balance > 1 && getBalance(root->left) < 0) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
    if (balance < -1 && getBalance(root->right) <= 0) return
leftRotate(root);
    if (balance < -1 && getBalance(root->right) > 0) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }
    return root;
}
// In-order traversal
void inorder(Node* root, FILE* out) {
    if (root) {
        inorder(root->left, out);
        fprintf(out, "%d ", root->key);
        inorder(root->right, out);
    }
}
```

```
void printInorder(Node* root) {
    FILE* temp = tmpfile();
    inorder(root, temp);
    rewind(temp);
    int x;
    while (fscanf(temp, "%d", &x) == 1) {
        printf("%d ", x);
    }
    printf("\n");
    fclose(temp);
}
// Main Program
int main() {
    Node* root = NULL;
    char inputFile[20];
    printf("Input file name: ");
    scanf("%s", inputFile);
//lets say input file contains 10 20 30 40 50
    FILE* in = fopen(inputFile, "r");
    if (!in) {
        printf("Error opening input file\n");
        return 1;
    }
    int val;
        char line[256];
        fgets(line, sizeof(line), in);
        while (fscanf(in, "%d", &val) != EOF)
        root = insert(root, val);
    fclose(in);
    printf("In-order traversal:\n");
    printInorder(root);
    int choice, key;
    while (1) {
        printf("1. Insert\n2. Delete\n3. Write to file and
exit\nChoice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Element: ");
                scanf("%d", &key);
                root = insert(root, key);
                printf("In-order traversal:\n");
                printInorder(root);
                break;
            case 2:
                printf("Element: ");
```

```
                scanf("%d", &key);
                    int found = 0;
                    root = deleteNode(root, key, &found);
                    if (found)
                        printf("Deleted %d\n", key);
                    else
                        printf("%d not found\n", key);
                    printf("In-order traversal:\n");
                    printInorder(root);
                    break;
            case 3:{
                    char outputFile[100];
                    printf("Output file name: ");
                    scanf("%s", outputFile);
                    FILE* out = fopen(outputFile, "w");
                    if (!out) {
                        printf("Error opening output file\n");
                        return 1;
                    }
                    inorder(root, out);
                    fprintf(out, "\n");
                    fclose(out);
                    printf("AVL Tree written to file\n");
                    return 0;}
                default:
                printf("Invalid choice\n");
        }
    }
}
```

input1.txt

```
5
1 2 3 4 5
```

input2.txt

```
6
12 24 36 48 60 72
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Input file name:
inputfile1.txt
Error opening input file

Test Case - 2
User Output
Input file name:
input1.txt
In-order traversal:
1 2 3 4 5
1. Insert
2. Delete
3. Write to file and exit
Choice:
1
Element:
10
In-order traversal:
1 2 3 4 5 10
1. Insert
2. Delete
3. Write to file and exit
Choice:
2
Element:
100
100 not found
In-order traversal:
1 2 3 4 5 10
1. Insert
2. Delete
3. Write to file and exit
Choice:
2

Element:
3
Deleted 3
In-order traversal:
1 2 4 5 10
1. Insert
2. Delete
3. Write to file and exit
Choice:
3
Output file name:
myoutput.txt
AVL Tree written to file

Test Case - 3
User Output
Input file name:
input2.txt
In-order traversal:
12 24 36 48 60 72
1. Insert
2. Delete
3. Write to file and exit
Choice:
1
Element:
108
In-order traversal:
12 24 36 48 60 72 108
1. Insert
2. Delete
3. Write to file and exit
Choice:
1
Element:
120
In-order traversal:
12 24 36 48 60 72 108 120
1. Insert

2. Delete
3. Write to file and exit
Choice:
1
Element:
144
In-order traversal:
12 24 36 48 60 72 108 120 144
1. Insert
2. Delete
3. Write to file and exit
Choice:
2
Element:
36
Deleted 36
In-order traversal:
12 24 48 60 72 108 120 144
1. Insert
2. Delete
3. Write to file and exit
Choice:
2
Element:
48
Deleted 48
In-order traversal:
12 24 60 72 108 120 144
1. Insert
2. Delete
3. Write to file and exit
Choice:
10
Invalid choice
1. Insert
2. Delete
3. Write to file and exit
Choice:
2
Element:
1440

1440 not found
In-order traversal:
12 24 60 72 108 120 144
1. Insert
2. Delete
3. Write to file and exit
Choice:
15
Invalid choice
1. Insert
2. Delete
3. Write to file and exit
Choice:
3
Output file name:
outputfile.txt
AVL Tree written to file

S.No: 2	Exp. Name: ***Operation of B-Tree - Insertion, Deletion and Search***	**Date: 2026-10-01**

Aim:

Write a C program to construct a B-Tree of a given minimum degree. First, insert a list of keys into the B-Tree and print the inorder traversal after all insertions. Then, delete a given set of keys from the B-Tree and again print the inorder traversal. Finally, search for a specific key in the current B-Tree and print whether it is found or not.

Input Format:

- The first line contains a single integer t, the minimum degree of the B-Tree.
- The second line contains an integer n, representing the number of keys to be inserted.
- The third line contains n space-separated integers — the keys to be inserted into the B-Tree.
- The fourth line contains an integer d, the number of keys to delete.
- The fifth line contains d space-separated integers — the keys to be deleted from the B-Tree.
- The sixth line contains a single integer k — the key to be searched in the B-Tree.

Output Format:

- The first line of output should display the inorder traversal of the B-Tree after all insertions, with each key separated by a space.
- The second line should display the inorder traversal of the B-Tree after all deletions, with each key separated by a space.
- The third line should print **Found** if the search key exists in the B-Tree. If the key is not present, print **Not Found** instead.

Note: Refer to the test cases for better understanding.

Source Code:

Btreemain.c

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct BTreeNode {
    int *keys;
    int t;                                  //Minimum degree of B-Tree
    struct BTreeNode **C;    //Minimum degree of B-Tree
    int n;                                      //Stores addresses of child
nodes
    bool leaf;
} BTreeNode;

// Create a new node
BTreeNode* createNode(int t, bool leaf) {
    BTreeNode* node = (BTreeNode*)malloc(sizeof(BTreeNode));
    node->t = t;
    node->leaf = leaf;
    node->keys = (int*)malloc((2 * t - 1) * sizeof(int));
    node->C = (BTreeNode**)malloc((2 * t) * sizeof(BTreeNode*));
    node->n = 0;
    return node;
}

// Inorder traversal (no trailing dot)
void inorder(BTreeNode* root) {
    if (!root) return;
    for (int i = 0; i < root->n; i++) {
        if (!root->leaf)
            inorder(root->C[i]);
        printf("%d ", root->keys[i]);
    }
    if (!root->leaf)
        inorder(root->C[root->n]);
}

// Search a key
bool search(BTreeNode* root, int k) {
    int i = 0;
    while (i < root->n && k > root->keys[i])
        i++;
    if (i < root->n && root->keys[i] == k)
        return true;
    if (root->leaf)
```

```
        return false;
    return search(root->C[i], k);
}

// Split full child
void splitChild(BTreeNode* x, int i) {
    int t = x->t;
    BTreeNode* y = x->C[i];
    BTreeNode* z = createNode(t, y->leaf);
    z->n = t - 1;
    for (int j = 0; j < t - 1; j++)
        z->keys[j] = y->keys[j + t];
    if (!y->leaf) {
        for (int j = 0; j < t; j++)
            z->C[j] = y->C[j + t];
    }
    y->n = t - 1;
    for (int j = x->n; j >= i + 1; j--)
        x->C[j + 1] = x->C[j];
    x->C[i + 1] = z;
    for (int j = x->n - 1; j >= i; j--)
        x->keys[j + 1] = x->keys[j];
    x->keys[i] = y->keys[t - 1];
    x->n++;
}

// Insert into non-full node
void insertNonFull(BTreeNode* x, int k) {
    int i = x->n - 1;
    if (x->leaf) {
        while (i >= 0 && k < x->keys[i]) {
            x->keys[i + 1] = x->keys[i];
            i--;
        }
        x->keys[i + 1] = k;
        x->n++;
    } else {
        while (i >= 0 && k < x->keys[i])
            i--;
        i++;
        if (x->C[i]->n == 2 * x->t - 1) {
            splitChild(x, i);
            if (k > x->keys[i])
                i++;
        }
        insertNonFull(x->C[i], k);
```

```
    }
}

// Insert key into tree
void insert(BTreeNode** rootRef, int k) {
    BTreeNode* r = *rootRef;
    if (r->n == 2 * r->t - 1) {
        BTreeNode* s = createNode(r->t, false);
        s->C[0] = r;
        splitChild(s, 0);
        int i = (k > s->keys[0]) ? 1 : 0;
        insertNonFull(s->C[i], k);
        *rootRef = s;
    } else {
        insertNonFull(r, k);
    }
}

// Get predecessor
int getPred(BTreeNode* x) {
    while (!x->leaf)
        x = x->C[x->n];
    return x->keys[x->n - 1];
}

// Get successor
int getSucc(BTreeNode* x) {
    while (!x->leaf)
        x = x->C[0];
    return x->keys[0];
}

// Merge two children
void merge(BTreeNode* x, int idx) {
    BTreeNode* c1 = x->C[idx];
    BTreeNode* c2 = x->C[idx + 1];
    int t = x->t;
    c1->keys[t - 1] = x->keys[idx];
    for (int i = 0; i < c2->n; ++i)
        c1->keys[i + t] = c2->keys[i];
    if (!c1->leaf) {
        for (int i = 0; i <= c2->n; ++i)
            c1->C[i + t] = c2->C[i];
    }
    for (int i = idx + 1; i < x->n; ++i)
        x->keys[i - 1] = x->keys[i];
```

```
    for (int i = idx + 2; i <= x->n; ++i)
        x->C[i - 1] = x->C[i];
    c1->n += c2->n + 1;
    x->n--;
    free(c2);
}

// Fill child if less than t-1 keys
void fill(BTreeNode* x, int idx) {
    int t = x->t;
    if (idx > 0 && x->C[idx - 1]->n >= t) {
        BTreeNode* child = x->C[idx];
        BTreeNode* sibling = x->C[idx - 1];
        for (int i = child->n - 1; i >= 0; --i)
            child->keys[i + 1] = child->keys[i];
        if (!child->leaf) {
            for (int i = child->n; i >= 0; --i)
                child->C[i + 1] = child->C[i];
        }
        child->keys[0] = x->keys[idx - 1];
        if (!child->leaf)
            child->C[0] = sibling->C[sibling->n];
        x->keys[idx - 1] = sibling->keys[sibling->n - 1];
        child->n += 1;
        sibling->n -= 1;
    } else if (idx < x->n && x->C[idx + 1]->n >= t) {
        BTreeNode* child = x->C[idx];
        BTreeNode* sibling = x->C[idx + 1];
        child->keys[child->n] = x->keys[idx];
        if (!child->leaf)
            child->C[child->n + 1] = sibling->C[0];
        x->keys[idx] = sibling->keys[0];
        for (int i = 1; i < sibling->n; ++i)
            sibling->keys[i - 1] = sibling->keys[i];
        if (!sibling->leaf) {
            for (int i = 1; i <= sibling->n; ++i)
                sibling->C[i - 1] = sibling->C[i];
        }
        child->n += 1;
        sibling->n -= 1;
    } else {
        if (idx != x->n)
            merge(x, idx);
        else
            merge(x, idx - 1);
    }
```

```
}

// Remove key from tree
void removeFromNode(BTreeNode* x, int k) {
    int idx = 0;
    while (idx < x->n && x->keys[idx] < k)
        ++idx;
    if (idx < x->n && x->keys[idx] == k) {
        if (x->leaf) {
            for (int i = idx + 1; i < x->n; ++i)
                x->keys[i - 1] = x->keys[i];
            x->n--;
        } else {
            if (x->C[idx]->n >= x->t) {
                int pred = getPred(x->C[idx]);
                x->keys[idx] = pred;
                removeFromNode(x->C[idx], pred);
            } else if (x->C[idx + 1]->n >= x->t) {
                int succ = getSucc(x->C[idx + 1]);
                x->keys[idx] = succ;
                removeFromNode(x->C[idx + 1], succ);
            } else {
                merge(x, idx);
                removeFromNode(x->C[idx], k);
            }
        }
    } else {
        if (x->leaf)
            return;
        bool flag = (idx == x->n);
        if (x->C[idx]->n < x->t)
            fill(x, idx);
        if (flag && idx > x->n)
            removeFromNode(x->C[idx - 1], k);
        else
            removeFromNode(x->C[idx], k);
    }
}

// Delete key wrapper
void deleteKey(BTreeNode** rootRef, int k) {
    BTreeNode* r = *rootRef;
    removeFromNode(r, k);
    if (r->n == 0) {
        BTreeNode* tmp = r;
        if (r->leaf)
```

```
            *rootRef = NULL;
        else
            *rootRef = r->C[0];
        free(tmp);
    }
}

// Main function with test case support
int main() {
    int t;
    scanf("%d", &t);
    BTreeNode* root = createNode(t, true);
    int n, val;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insert(&root, val);
    }
    inorder(root);
    int d;
    scanf("%d", &d);
    for (int i = 0; i < d; i++) {
        scanf("%d", &val);
        deleteKey(&root, val);
    }
    inorder(root);
    int target;
    scanf("%d\n", &target);
    if (search(root, target))
        printf("Found\n");
    else
        printf("Not Found\n");

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
2
7

10 20 5 6 12 30 7
3
6 30 100
12
5 6 7 10 12 20 30
5 7 10 12 20
Found

Test Case - 2
User Output
2
6
8 3 10 1 6 14
2
1 3
2
1 3 6 8 10 14
6 8 10 14
Not Found

S.No: 3	Exp. Name: ***Min Heap Operations***	**Date: 2026-10-03**

Aim:

Write a C program to implement a Min Heap of integers. The program should first read a list of integers and insert them into the heap while maintaining the min-heap property. It should then read the number of deletions to perform, delete the root (minimum element) that many times, and display the heap both before and after the deletions.

Input Format:

- The first line contains an integer n, representing the number of elements to insert into the heap.
- The second line contains n space-separated integers — the values to insert into the heap.
- The third line contains an integer k, representing the number of times the root (minimum element) should be deleted from the heap.

Output Format:

- The first line displays the space-separated heap contents after all insertions.
- The second line displays the space-separated heap contents after performing k deletions of the root element.

Note: Refer to the test cases for better understanding.

Source Code:

minheap.c

```
#include <stdio.h>

#define MAX_SIZE 20

int heap[MAX_SIZE];
int size = 0;

// Swap utility
void swap(int *a, int *b) {
    int temp = *a;
    *a=*b;
    *b=temp;
}

// Iterative heapify up
void heapifyUp(int index) {
    while (index>0) {
        int parent=(index-1)/2;
        if(heap[index]<heap[parent]) {
            swap(&heap[index], &heap[parent]);
            index = parent;
        }
                else{
            break;
        }
    }
}

// Insert into min heap
void insert(int value) {
    heap[size] = value;
    heapifyUp(size);
    size++;
}

// Iterative heapify down
void heapifyDown(int index) {
    while(1){
        int left=2*index+1;
        int right=2*index+2;
        int smallest=index;
        if (left<size&&heap[left]<heap[smallest])
            smallest=left;
        if(right<size&&heap[right]<heap[smallest])
```

```
            smallest=right;
                if (smallest != index) {
                swap(&heap[index],&heap[smallest]);
            index = smallest;
        }
                else{
            break;
        }
    }
}

// Delete root (minimum element)
void deleteMin() {
    if(size==0)
                return;
            heap[0] = heap[size - 1];
            size--;
            heapifyDown(0);
}

// Print heap
void printHeap() {
    for(int i=0;i<size;i++){
        printf("%d ",heap[i]);
    }
}

int main() {
    int n,del,val;

    // Read number of elements
        scanf("%d",&n);

    // Insert elements into heap
    for (int i=0;i<n;i++){
                scanf("%d",&val);
            insert(val);

    }

    // Print heap after insertions
    printHeap();
        scanf("%d\n",&del);
    for (int i=0;i<del;i++){
        deleteMin();
    }
```

```
    // Print heap after deletions
    printHeap();
    printf("\n");

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
20 15 30 5 10
2
5 10 30 20 15
15 20 30

Test Case - 2
User Output
8
70 10 90 20 60 80 30 40
4
10 20 30 40 60 90 80 70
60 70 80 90

S.No: 4	Exp. Name: ***Max Heap Operations***	Date: 2026-10-03

Aim:

Write a C program to implement a Max Heap. Insert a given list of integers into the heap while maintaining the max-heap property. Then, delete the root element (i.e., the maximum element) a specified number of times. Display the heap after all insertions and again after the deletions.

Input Format:

- The first line contains an integer n, the number of elements to insert into the heap.
- The second line contains n space-separated integers representing the values to be inserted.
- The third line contains an integer k, the number of times the root element (maximum) should be deleted.

Output Format:

- The first line displays the space-separated heap contents after all insertions.
- The second line displays the space-separated heap contents after performing k deletions of the root element.

Note: Refer to the test cases for better understanding.

Source Code:

maxheap.c

```
#include<stdio.h>
#define MAX_SIZE 20

int heap[MAX_SIZE];
int size=0;

void swap(int *a,int *b){
    int temp=*a;
    *a=*b;
    *b=temp;
}
void heapifyUp(int index){
    while(index>0){
        int parent=(index-1)/2;
        if(heap[index]>heap[parent]){
            swap(&heap[index],&heap[parent]);
            index=parent;
        }
        else{
            break;
        }
    }
}
void insert(int value){
    heap[size]=value;
    heapifyUp(size);
    size++;
}
void heapifyDown(int index){
    while(1){
        int left=2*index+1;
        int right=2*index+2;
        int smallest=index;

        if(left<size&&heap[left]>heap[smallest])
                        smallest=left;

        if(right<size&&heap[right]>heap[smallest])
            smallest=right;

if(smallest!=index){
                        swap(&heap[index],&heap[smallest]);
            index=smallest;
        }
```

```
        else{
            break;
        }
    }
}
void deleteMax(){
    if(size==0)
        return;
            heap[0]=heap[size-1];
            size--;
            heapifyDown(0);
}
void printHeap(){
    for(int i=0;i<size;i++){
        printf("%d ",heap[i]);
    }
}
int main(){
    int n,del,val;
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&val);
        insert(val);
    }
    printHeap();
    scanf("%d\n",&del);
    for(int i=0;i<del;i++){
        deleteMax();
    }
    printHeap();
    printf("\n");
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
6
30 10 20 40 50 5
2

50 40 20 10 30 5
30 10 20 5

Test Case - 2
User Output
4
8 3 6 2
1
8 3 6 2
6 3 2

S.No: 5	Exp. Name: ***Depth First Search (DFS) on a Graph***	**Date: 2026-10-01**

Aim:

Write a program to perform Depth First Search (DFS) traversal on an undirected graph using an adjacency matrix representation. The program should take the number of vertices, edges, and the starting vertex as input and print the DFS traversal of the graph.

Input Format:

- The first line contains an integer N, representing the number of vertices in the graph.
- The second line contains an integer E, representing the number of edges in the graph.
- The next E lines contain two integers v_1, v_2 representing an edge between vertices v_1 and v_2.
- The last line contains an integer S, representing the starting vertex for the DFS traversal.

Output Format:

- Print the DFS traversal of the graph starting from S, with vertices separated by a space.

Note:

- If multiple nodes are available to visit, DFS should proceed in the order of their appearance in the adjacency matrix.

Source Code:

CTC64641.c

```
#include <stdio.h>
#define MAX 10
int adj[MAX][MAX]; // adjacency matrix
int visited[MAX];
int stack[MAX];
int top = -1;
int n; // number of vertices
void push(int v) {
    stack[++top] = v;
}
int pop() {
    return stack[top--];
}
int isEmpty() {
    return (top == -1);
}
void dfs_stack(int start) {
    push(start);
    visited[start] = 1; // mark early

    while (!isEmpty()) {
        int current = pop();
        printf("%d ", current);
        for (int i = n ; i >= 0; i--) {
            if (adj[current][i] == 1 && !visited[i]) {
                push(i);
                visited[i] = 1; // mark here to avoid duplicate pushes
            }
        }
    }
}
int main() {
    int edges, u, v, start;
    // Input vertices and edges
    scanf("%d", &n);
    scanf("%d", &edges);
    // Initialize adjacency matrix and visited array
    for (int i = 0; i < n; i++) {
        visited[i] = 0;
        for (int j = 0; j < n; j++) {
            adj[i][j] = 0;
        }
    }
    // Read edges
```

```
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1; // undirected graph
    }
    // Read starting vertex
    scanf("%d", &start);
    // DFS traversal using stack
    dfs_stack(start);
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
4
0 1
0 2
1 2
2 3
0
0 1 2 3

Test Case - 2
User Output
6
4
0 1
0 2
3 4
4 5
0
0 1 2

S.No: 6	Exp. Name: ***Breadth First Search (BFS) on a Graph***	**Date: 2026-10-01**

Aim:

Write a program to perform Breadth First Search (BFS) traversal on an undirected graph using an adjacency matrix representation. The program should take the number of vertices, edges, and the starting vertex as input and print the BFS traversal of the graph.

Input Format:

- The first line contains an integer N, representing the number of vertices in the graph.
- The second line contains an integer E, representing the number of edges in the graph.
- The next E lines contain two space-separated integers v_1, v_2 representing an edge between vertices v_1 and v_2.
- The last line contains an integer S, representing the starting vertex for the BFS traversal.

Output Format:

- Print the BFS traversal of the graph starting from S, with vertices separated by a space. If a vertex has multiple neighbors, process them in increasing order.

Source Code:

CTC64721.c

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX 10
int adj[MAX][MAX];   // Adjacency matrix
int visited[MAX];    // Visited array
int queue[MAX];      // Queue for BFS
int front = 0, rear = 0;
int n;               // Number of vertices
// Enqueue function
void enqueue(int vertex) {
    queue[rear++] = vertex;
}
// Dequeue function
int dequeue() {
    return queue[front++];
}
// Check if queue is empty
int isEmpty() {
    return front == rear;
}
// Breadth First Search function
void bfs(int start) {
    visited[start] = 1;
    enqueue(start);
    while (!isEmpty()) {
        int current = dequeue();
        printf("%d ", current);
        // Traverse neighbors in increasing order
        for (int i = 0; i < n; i++) {
            if (adj[current][i] == 1 && !visited[i]) {
                visited[i] = 1;
                enqueue(i);
            }
        }
    }
}
int main() {
    int edges, u, v, start;
    // Input number of vertices
    scanf("%d", &n);
    // Input number of edges
    scanf("%d", &edges);
    // Initialize adjacency matrix and visited array
    for (int i = 0; i < n; i++) {
```

```
        visited[i] = 0;
        for (int j = 0; j < n; j++) {
            adj[i][j] = 0;
        }
    }
    // Read edges and update matrix
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1; // Undirected graph
    }
    // Input starting vertex
    scanf("%d", &start);
    // Perform BFS
    bfs(start);
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
4
0 1
0 2
1 2
2 3
0
0 1 2 3

Test Case - 2
User Output
6
4
0 1
0 2

3 4
4 5
0
0 1 2

Test Case - 3
User Output
1
0
0
0

S.No: 7	Exp. Name: ***Depth-First Search (DFS)***	**Date: 2026-10-01**

Aim:

Write a C program to implement and perform a Depth-First Search (DFS) on a directed graph represented using an adjacency list. The program should print all the nodes reachable from a given starting node.

Input Format:

- The first line contains two integers: V (number of vertices) and E (number of edges).
- The next E lines each contain two integers: u and v, where there is a directed edge from node u to node v.
- The last line contains the starting node for the DFS traversal.

Output Format:

- Print all reachable nodes from the starting node in the order they are visited by DFS as a single line of space-separated integers.

Note:

- In DFS, the smaller numbered node is visited first when there are multiple adjacent nodes, ensuring a consistent traversal order
- Refer to the visible test cases for a better understanding.

Source Code:

dfs_directed_graph.c

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int adj[MAX][MAX];  // Adjacency matrix-style list
int visited[MAX];
int adjCount[MAX];  // Number of adjacent vertices for each node
// Comparison function for qsort
int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}
void dfs(int vertex) {
    visited[vertex] = 1;
    printf("%d ", vertex);
    for (int i = 0; i < adjCount[vertex]; i++) {
        int neighbor = adj[vertex][i];
        if (!visited[neighbor]) {
            dfs(neighbor);
        }
    }
}
int main() {
    int vertices, edges;
    scanf("%d %d", &vertices, &edges);
    // Initialize adjacency list and visited array
    for (int i = 0; i < vertices; i++) {
        adjCount[i] = 0;
        visited[i] = 0;
    }
    // Read edges
    for (int i = 0; i < edges; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][adjCount[u]++] = v; // directed edge u → v
    }
    // Sort neighbors to process in increasing order
    for (int i = 0; i < vertices; i++) {
        qsort(adj[i], adjCount[i], sizeof(int), compare);
    }
    int start;
    scanf("%d", &start);
    dfs(start);
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5 6
0 1
0 2
1 3
1 4
2 4
3 4
0
0 1 3 4 2

Test Case - 2
User Output
5 5
0 1
0 4
1 2
2 3
3 4
0
0 1 2 3 4

S.No: 8	Exp. Name: ***Breadth First Search (BFS)***	**Date: 2026-10-01**

Aim:

Write a C program to perform BFS on a directed graph using adjacency lists. The graph has V vertices and E edges. After building the graph, sort each vertex's adjacency list in ascending order. Then perform BFS from a given starting node and print all reachable nodes in the order they are visited as space-separated integers. When multiple adjacent nodes are available, always visit the smallest one first.

Input Format:

- The input begins with a line containing two integers V and E, where V is the number of vertices and E is the number of edges in the directed graph.
- The following E lines each contain two integers u and v, indicating a directed edge from vertex u to vertex v.
- The final line contains a single integer $start$, which represents the starting vertex for the BFS traversal.

Output Format:

- Print all reachable nodes from the starting vertex in the order they are visited by BFS, as a single line of space-separated integers.

Note: Refer to the test cases for better understanding.

Source Code:

BFS.c

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int adj[MAX][MAX];     // Adjacency list
int adjCount[MAX];     // Number of neighbors for each node
int visited[MAX];      // Visited array
int queue[MAX];        // Queue for BFS
int front = 0, rear = 0;
// Comparison function for sorting neighbors
int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}
// Enqueue operation
void enqueue(int v) {
    queue[rear++] = v;
}
// Dequeue operation
int dequeue() {
    return queue[front++];
}
// Check if queue is empty
int isEmpty() {
    return front == rear;
}
// BFS function
void bfs(int start) {
    visited[start] = 1;
    enqueue(start);
    while (!isEmpty()) {
        int current = dequeue();
        printf("%d ", current);
        for (int i = 0; i < adjCount[current]; i++) {
            int neighbor = adj[current][i];
            if (!visited[neighbor]) {
                visited[neighbor] = 1;
                enqueue(neighbor);
            }
        }
    }
}
int main() {
    int vertices, edges;
    scanf("%d %d", &vertices, &edges);
    // Initialize structures
```

```
    for (int i = 0; i < vertices; i++) {
        adjCount[i] = 0;
        visited[i] = 0;
    }
    // Input edges
    for (int i = 0; i < edges; i++) {
        int u, v;
        scanf("%d %d", &u, &v); // directed edge u → v
        adj[u][adjCount[u]++] = v;
    }
    // Sort neighbors for each vertex
    for (int i = 0; i < vertices; i++) {
        qsort(adj[i], adjCount[i], sizeof(int), compare);
    }
    int start;
    scanf("%d", &start);
    // Perform BFS
    bfs(start);
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5 4
0 1
0 2
1 3
1 4
0
0 1 2 3 4

Test Case - 2
User Output
4 6
0 1
0 2

0 3
1 2
1 3
2 3
0
0 1 2 3

S.No: 9	Exp. Name: ***Bi-Connected Components in Graph***	**Date: 2026-11-01**

Aim:

Write a C program for finding the count of bi-connected components in a given graph

Input Format:

- First line in the input indicates the number of vertices.
- Second line in the input indicates the number of edges.
- The next following lines indicates the edges, which takes two vertices u v as space separated integers for each edge.

Output Format:

- The code will print the number of biconnected components found in the graph.

Note: Refer to the test cases for better understanding.

Source Code:

biConnected.c

```
// write your code here...
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int V, E;
int adj[MAX][MAX];
int disc[MAX], low[MAX], parent[MAX];
int time_dfs, bccCount;

struct Edge {
    int u, v;
} stackEdges[MAX * MAX];

int top = -1;

void pushEdge(int u, int v) {
    stackEdges[++top] = (struct Edge){u, v};
}

void popUntil(int u, int v) {
    while (top >= 0) {
        struct Edge e = stackEdges[top--];
        if ((e.u == u && e.v == v) || (e.u == v && e.v == u))
            break;
    }
}

void BCCUtil(int u) {
    int children = 0;
    disc[u] = low[u] = ++time_dfs;

    for (int v = 0; v < V; v++) {
        if (adj[u][v]) {
            if (disc[v] == -1) {
                children++;
                parent[v] = u;
                pushEdge(u, v);
                BCCUtil(v);

                if (low[v] >= disc[u]) { // found a BCC
                    bccCount++;
                    popUntil(u, v);
```

```
                }
                if (low[v] < low[u])
                    low[u] = low[v];
            }
            else if (v != parent[u] && disc[v] < disc[u]) {
                pushEdge(u, v);
                if (disc[v] < low[u])
                    low[u] = disc[v];
            }
        }
    }
}

void findBCCs() {
    for (int i = 0; i < V; i++) {
        disc[i] = -1;
        parent[i] = -1;
    }
    time_dfs = 0;
    bccCount = 0;
    top = -1;

    for (int i = 0; i < V; i++) {
        if (disc[i] == -1) {
            BCCUtil(i);
            if (top >= 0) { // leftover edges form a BCC
                bccCount++;
                top = -1;
            }
        }
    }
}

int main() {
    scanf("%d %d", &V, &E);

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            adj[i][j] = 0;

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = adj[v][u] = 1;
    }
```

```
    findBCCs();
    printf("%d\n", bccCount);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3
2
0 1
0 2
2

Test Case - 2
User Output
5
4
0 1
1 2
1 3
2 4
4

S.No: 10	Exp. Name: ***C program that implements the quick sort***	**Date: 2026-10-03**

Aim:

Write a C program that implements the quick sort to sort a given list of integers in ascending order.

Input Format

- The first line contains an integer n which represents the number of elements in the array.
- The second line contains n space-separated integers, which are the elements of the array.

Output Format

- A single line containing n space-separated integers, representing the array elements in ascending order after sorting.

Note: Refer to the test cases for better understanding.

Source Code:

quickSort.c

```
// write your code here...
// write your code here...
#include<stdio.h>
void swap(int *a,int *b){
    int temp=*a;
    *a=*b;
    *b=temp;
}
int partition(int a[],int low,int high){
    int pi=a[low];
    int i=low;
    int j=high+1;
    while(i<j){
        if(i<j){
            do{
                i++;
            }
            while(a[i]<=pi&&i<=high);
            do{
                j--;
            }
            while(a[j]>pi);
        }
        if(i<j){
            swap(&a[i],&a[j]);
        }
    }
    swap(&a[low],&a[j]);
    return j;
}
void quicksort(int a[],int low,int high){
    if(low<high){
        int pi=partition(a,low,high);
        quicksort(a,low,pi-1);
        quicksort(a,pi+1,high);
    }
}
int main()
{
    int i,n;
    scanf("%d",&n);
    int a[n];
    for(i=0;i<n;i++){
        scanf("%d",&a[i]);
```

```
    }
    quicksort(a,0,n-1);
    for(i=0;i<n;i++){
        printf("%d ",a[i]);
    }
    printf("\n");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
6
6 2 8 10 36 14
2 6 8 10 14 36

Test Case - 2
User Output
8
95 14 10 23 36 2 5 35
2 5 10 14 23 35 36 95

S.No: 11	Exp. Name: ***C program that implements the merge sort***	**Date: 2026-10-03**

Aim:

Write a C program to implement the Merge Sort algorithm to sort an array of integers in ascending order.

Input Format

- The first line contains an integer n which represents the number of elements in the array.
- The second line contains n space-separated integers, which are the elements of the array.

Output Format

- A single line containing n space-separated integers, representing the array elements in ascending order after sorting.

Note: Refer to the test cases for better understanding.

Source Code:

mergeSort.c

```
// write your code here...
// write your code here...
#include<stdio.h>

void merge(int a[], int low, int mid, int high){
    int temp[10];
    int i = low;
    int j = mid + 1;
    int k = low;

    while(i <= mid && j <= high){
        if(a[i] <= a[j]){
            temp[k++] = a[i++];
        }
        else{
            temp[k++] = a[j++];
        }
    }

    while(i <= mid){
        temp[k++] = a[i++];
    }

    while(j <= high){
        temp[k++] = a[j++];
    }

    for(i = low; i <= high; i++){
        a[i] = temp[i];
    }
}

void mergeSort(int a[], int low, int high){
    if(low < high){
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

int main(){
    int i, n;
    scanf("%d", &n);
```

```
    int a[n];

    for(i = 0; i < n; i++){
        scanf("%d", &a[i]);
    }

    mergeSort(a, 0, n - 1);

    for(i = 0; i < n; i++){
        printf("%d ", a[i]);
    }

    printf("\n");
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
50 20 30 40 10
10 20 30 40 50

Test Case - 2
User Output
10
25 78 15 45 32 65 46 38 76 12
12 15 25 32 38 45 46 65 76 78

S.No: 12	Exp. Name: ***Shortest Path from Source to Vertex***	**Date: 2026-10-14**

Aim:

You are required to write a C program that computes the shortest paths from a source vertex to all other vertices in a weighted, directed graph. The program should utilize Dijkstra's algorithm to solve this problem.

Input Format:

- The first line contains two integers V and E, representing the number of vertices and the number of edges respectively.
- The next E lines each contain three integers u, v, and w, representing a directed edge from vertex u to vertex v with weight w.
- The last line contains a single integer src, representing the source vertex from which shortest distances are to be calculated.

Output Format:

- The output consists of V lines.
- Each line contains two integers:
    - The vertex number and the minimum distance from the source vertex to that vertex, space separated

Note: Refer to the test cases for better understanding.

Source Code:

Path.c

```
// write your code here...
#include <stdio.h>
#include <limits.h>
#define MAX 100
#define INF INT_MAX

int n, m;
int graph[MAX][MAX];

int minDistance(int dist[], int visited[]) {
    int min = INF, min_index = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }

    return min_index;
}

void dijkstra(int src) {
    int dist[MAX];
    int visited[MAX];

    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {
        int u = minDistance(dist, visited);

        if (u == -1)
            break;

        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && graph[u][v] != INF &&
                dist[u] != INF &&
```

```
                dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    for (int i = 0; i < n; i++) {
        if (dist[i] == INF)
            printf("%d -1\n", i);
        else
            printf("%d %d\n", i, dist[i]);
    }
}

int main() {
    scanf("%d %d", &n, &m);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            graph[i][j] = INF;

    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        graph[u][v] = w;
    }

    int src;
    scanf("%d", &src);

    dijkstra(src);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4 3
0 1 2
0 2 5

0 3 1
0
0 0
1 2
2 5
3 1

Test Case - 2
User Output
6 9
0 1 7
0 2 9
0 5 14
1 2 10
1 3 15
2 3 11
2 5 2
3 4 6
5 4 9
0
0 0
1 7
2 9
3 20
4 20
5 11

S.No: 13	Exp. Name: ***Job Sequencing using Greedy Problem***	**Date: 2026-10-14**

Aim:

Given a set of$N$jobs, where eachjob $i$has a deadline and a profit associated with it. Each job takes1unit of time to complete, and only one job can be scheduled at a time. We earn the profit associated with the job if and only if the job is completed by its deadline. Find the number of jobs done and themaximum profit.

Note: Jobs will be given in the form (Job Id, Deadline, Profit) associated with that Job. The deadline of the job is the time by which the job needs to be completed to earn the profit.

Input Format:

- The first line contains an integer N representing the number of jobs.
- The next N lines each contain three space-separated integers: {Job ID, Job Deadline, Job Profit} — representing one job per line.

Output Format

- Two integers separated by a space:
- The first integer represents the number of jobs done.The second integer represents the maximum profit obtained.

Constraints:

- $1 \leq N \leq 10^5$
- $1 \leq Deadline \leq N$
- $1 \leq Profit \leq 500$

Example 1:

Input:

```
4
1 4 20
2 1 10
3 1 40
4 1 30
```

Output:

```
2 60
```

Explanation: Job 1and Job 3 can be done with a maximum profit of 60 (20+40).

Note: Refer to the test cases for better understanding.

Source Code:

JsGreedy.c

```
// write your code here...

#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id;
    int deadline;
    int profit;
} Job;

int compare(const void *a, const void *b) {
    Job *j1 = (Job *)a;
    Job *j2 = (Job *)b;
    return j2->profit - j1->profit;
}

int main() {
    int n;
    scanf("%d", &n);

    Job jobs[n];

    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &jobs[i].id, &jobs[i].deadline,
&jobs[i].profit);
    }

    qsort(jobs, n, sizeof(Job), compare);

    int maxDeadline = 0;

    for (int i = 0; i < n; i++) {
        if (jobs[i].deadline > maxDeadline) {
            maxDeadline = jobs[i].deadline;
        }
    }

    int slot[maxDeadline + 1];

    for (int i = 0; i <= maxDeadline; i++) {
        slot[i] = -1;
    }
```

```
    int countJobs = 0;
    int totalProfit = 0;

    for (int i = 0; i < n; i++) {
        for (int j = jobs[i].deadline; j > 0; j--) {
            if (slot[j] == -1) {
                slot[j] = jobs[i].id;
                countJobs++;
                totalProfit += jobs[i].profit;
                break;
            }
        }
    }

    printf("%d %d\n", countJobs, totalProfit);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
1 4 20
2 1 10
3 1 40
4 1 30
2 60

Test Case - 2
User Output
5
1 2 60
2 1 100
3 3 20
4 4 70

5 2 50
4 250

S.No: 14	Exp. Name: ***Program to implement Knapsack problem Using Dynamic Programming***	**Date: 2026-10-26**

Aim:

You are given n items, each with a specific weight and value. You have a knapsack with a maximum capacity W. Your goal is to select a subset of these items such that the total weight is less than or equal to W, and the total value is as large as possible.

This is the classic 0/1 Knapsack problem, where each item can either be included once or excluded from the knapsack.

Input Format:

- The first line contains an integer n, representing the number of items to consider for the knapsack.
- The second line contains n space-separated integers, representing the weights of the items.
- The third line contains n space-separated integers, representing the values of the items, corresponding to their weights.
- The fourth line contains an integer W, representing the capacity of the knapsack.

Output Format:

- Print the maximum profit that can be obtained by selecting items such that their combined weight does not exceed the knapsack capacity.

Note: Refer to the test cases for better understanding.

Source Code:

dynamicKs.c

```
// write your code here...

#include <stdio.h>

int knapsack(int weights[], int profits[], int n, int m) {
    int M[n + 1][m + 1];

    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            if (i == 0 || j == 0)
                M[i][j] = 0;
            else if (weights[i - 1] <= j)
                M[i][j] = (profits[i - 1] + M[i - 1][j - weights[i -
1]] > M[i - 1][j]) ? (profits[i - 1] + M[i - 1][j - weights[i - 1]]) :
M[i - 1][j];
            else
                M[i][j] = M[i - 1][j];
        }
    }

    return M[n][m];
}

int main() {
    int n;
    scanf("%d", &n);

    int weights[n], profits[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &weights[i]);
    }

    for (int i = 0; i < n; i++) {
        scanf("%d", &profits[i]);
    }

    int capacity;
    scanf("%d", &capacity);

    int maxProfit = knapsack(weights, profits, n, capacity);

    printf("%d\n", maxProfit);
```

```
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
5 10 15 20 25
10 20 30 40 50
500
150

Test Case - 2
User Output
4
100 200 300 400
10 20 30 40
100
10

S.No: 15	Exp. Name: ***N-Queens Problem***	**Date: 2026-11-03**

Aim:

Write a C program to solve the N-Queens problem using backtracking. The goal is to place N queens on an $N \times N$ chessboard such that no two queens attack each other. A queen can attack another queen if they share the same row, column, or diagonal. Your program should print the first valid configuration of the board found by placing queens from left to right and top to bottom, using a backtracking approach. The board must be displayed as an $N \times N$ matrix where a cell contains 1 if a queen is placed there, and 0 otherwise. If no solution exists for the given value of N, print a message indicating that no solution exists.

Input Format:

- A single integer N — the number of queens and the size of the chessboard.

Output Format:

- If a solution exists, print an $N \times N$ matrix consisting of 0s and 1s, where 1 represents a queen.
- If no solution exists, print: "No solution exists for N = <value>".
- Each row of the matrix should be printed on a new line, with space-separated values.

Note: Refer to the test cases for better understanding.

Source Code:

nqueens.c

```
// write your code here...
#include <stdio.h>
#include <stdbool.h>

#define MAX 20 // Maximum board size

int board[MAX][MAX];

// Function to print the solution board
void printSolution(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

// Function to check if placing a queen is safe
bool isSafe(int row, int col, int n) {
    int i, j;

    // Check this column on upper side
    for (i = 0; i < row; i++) {
        if (board[i][col] == 1) return false;
    }

    // Check upper left diagonal
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j] == 1) return false;
    }

    // Check upper right diagonal
    for (i = row, j = col; i >= 0 && j < n; i--, j++) {
        if (board[i][j] == 1) return false;
    }

    return true;
}

// Recursive utility function to solve N-Queens
bool solveNQueens(int row, int n) {
    if (row == n) {
        // All queens placed successfully
```

```
        return true;
    }

    for (int col = 0; col < n; col++) {
        if (isSafe(row, col, n)) {
            board[row][col] = 1;

            if (solveNQueens(row + 1, n)) {
                return true; // Found first valid solution
            }

            board[row][col] = 0; // Backtrack
        }
    }

    return false;
}

int main() {
    int n;
    scanf("%d", &n);

    if (n <= 0 || n > MAX) {
        printf("Invalid board size. Please use N between 1 and %d.\n",
MAX);
        return 0;
    }

    // Initialize board with 0s
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            board[i][j] = 0;
        }
    }

    if (solveNQueens(0, n)) {
        printSolution(n);
    } else {
        printf("No solution exists for N = %d\n", n);
    }

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Test Case - 2
User Output
3
No solution exists for N = 3

S.No: 16	Exp. Name: ***Knapsack using backtracking***	**Date: 2026-11-03**

Aim:

Write a C program to solve the 0/1 Knapsack Problem using backtracking approach. The program aims to maximize the total value of items that can be included in a knapsack without exceeding its capacity

Input Format:

- The first line contains an integer n, representing the number of items.
- The next n lines each contain two space-separated integers: the value and weight of an item.
- The last line contains an integer W, representing the capacity of the knapsack.

Output Format:

- Print the maximum profit (total value) that can be obtained by selecting items, considering the knapsack's capacity constraint.

Note: Refer to the test cases for better understanding.

Source Code:

knapsackBacktracking.c

```
#include <stdio.h>
int n;          // Number of items
int W;          // Knapsack capacity
int maxProfit = 0; // Maximum profit
int values[100], weights[100]; // Arrays to store values and weights

// Backtracking function
void knapsack(int i, int currentWeight, int currentValue) {
    if (i == n) { // All items considered
        if (currentValue > maxProfit)
            maxProfit = currentValue;
        return;
    }

    // Option 1: Include the current item if it doesn't exceed capacity
    if (currentWeight + weights[i] <= W) {
        knapsack(i + 1, currentWeight + weights[i], currentValue +
values[i]);
    }

    // Option 2: Exclude the current item
    knapsack(i + 1, currentWeight, currentValue);
}

int main() {
    // Read number of items
    scanf("%d", &n);

    // Read values and weights
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &values[i], &weights[i]);
    }

    // Read knapsack capacity
    scanf("%d", &W);

    // Call backtracking function starting from first item
    knapsack(0, 0, 0);

    // Print the maximum profit
    printf("%d\n", maxProfit);

    return 0;
```

```
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3
60 10
100 20
120 30
50
220

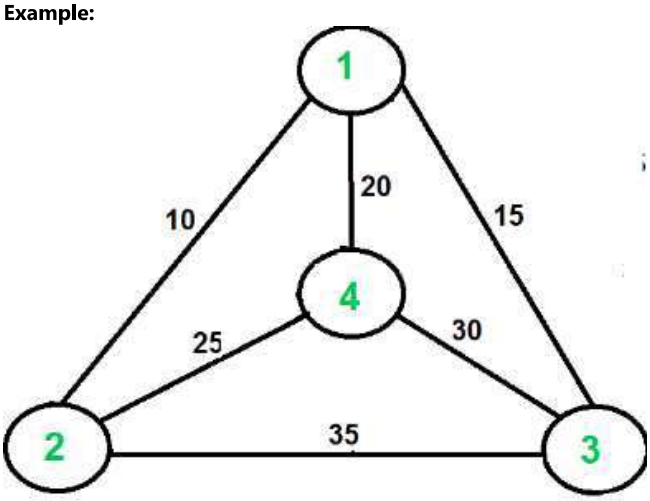
Test Case - 2
User Output
5
10 5
40 4
30 3
50 2
20 1
100
150

S.No: 17	Exp. Name: ***Travelling Salesman Problem***	**Date: 2026-11-03**

Aim:

Given a set of cities and the distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point.

Example:



A TSP tour in the graph is 1-2-4-3-1. The cost of the tour is 10+25+30+15, which is 80

Input Format:

- The first line contains an integer n, representing the number of cities.
- The following n lines contain the cost matrix of size $n \times n$. Each line contains n integers separated by spaces, representing the cost of traveling from one city to another. If there is no direct connection between two cities, -1 is used to represent infinity.

Output Format:

- The output is a single integer representing the optimal cost of the traveling salesman problem.

Source Code:

CTC30966.c

```
#include <stdio.h>
#include <limits.h>

int n;                  // Number of cities
int cost[20][20];       // Cost matrix
int visited[20];        // Visited cities
int minCost = INT_MAX;

// Backtracking function
void tsp(int city, int count, int currentCost, int start) {
    if (count == n) {
        // Return to starting city if path exists
        if (cost[city][start] != -1 && currentCost + cost[city][start]
< minCost) {
            minCost = currentCost + cost[city][start];
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[city][i] != -1) {
            visited[i] = 1;
            tsp(i, count + 1, currentCost + cost[city][i], start);
            visited[i] = 0;
        }
    }
}

int main() {
    // Read number of cities
    scanf("%d", &n);

    // Read cost matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    // Initialize visited array
    for (int i = 0; i < n; i++) visited[i] = 0;

    // Start TSP from city 0
    visited[0] = 1;
```

```
    tsp(0, 1, 0, 0);

    // Print minimum cost
    printf("%d\n", minCost);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3
-1 2 3
4 -1 6
1 2 -2
9

Test Case - 2
User Output
4
-1 2 3 4
1 -1 1 3
4 5 -1 1
2 3 6 -1
6